Retail Order Dispatch and Return Analysis

Python Programming – Activity-Based Learning (ABL) Assignment

1. Problem Statement

Process retail order quantities, dispatch records, and returned quantities; calculate outstanding orders, update inventory, and detect inconsistent records.

2. Objectives

• Store order details.

• Validate dispatch quantities against ordered quantities.

• Validate returns against dispatched quantities.

• Calculate outstanding and net-shipped quantities.

• Update inventory and report inconsistent records.

3. Assumptions

Each order has a unique order ID, item, ordered quantity, and unit price. Dispatches exceeding the remaining ordered quantity are rejected. Returns must not exceed accepted dispatched quantities. Inventory decreases for accepted dispatches and increases for accepted returns. The sample data deliberately includes inconsistent records for testing.

4. Algorithm

1. Store order records, dispatch records, return records, and stock.

2. Initialize dispatched and returned totals for each order.

3. Validate each dispatch ID and quantity.

4. Reject dispatches that would exceed the ordered quantity.

5. Validate return IDs and ensure returns do not exceed dispatches.

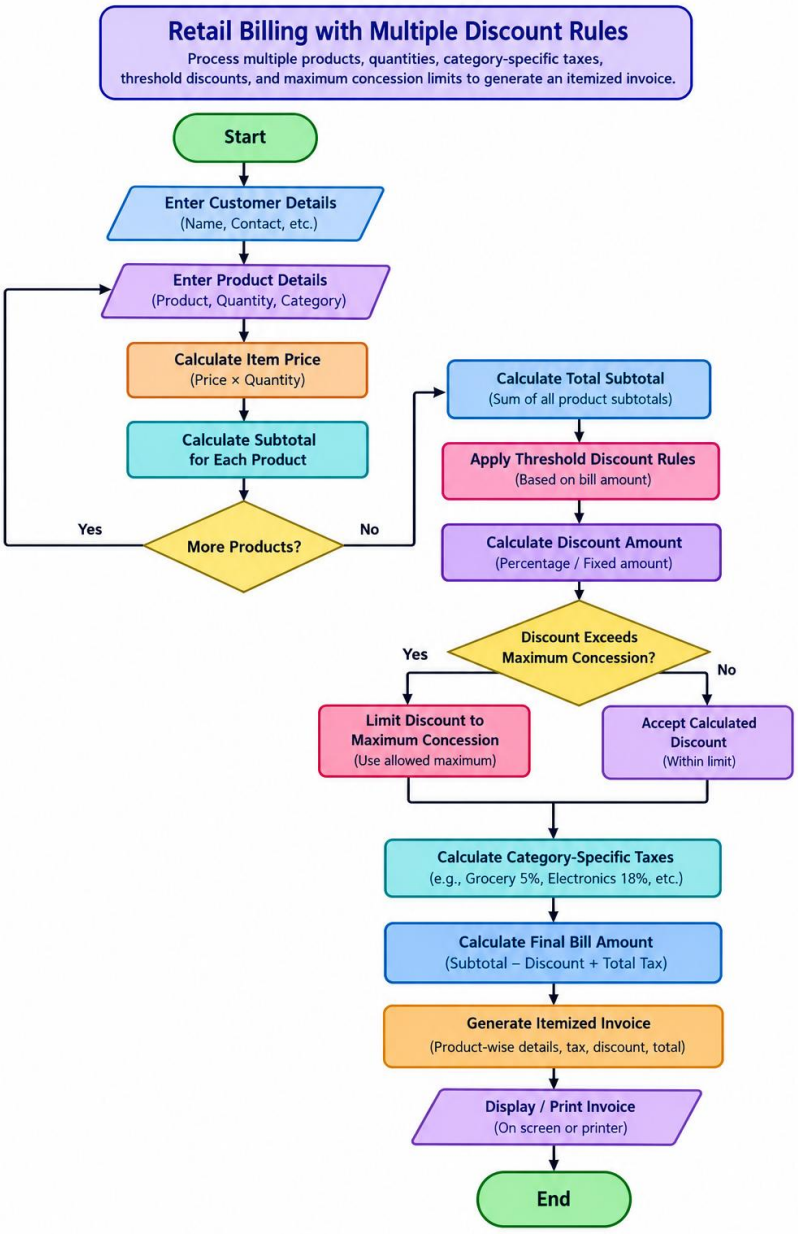
6. Update stock for accepted dispatches and returns.

7. Calculate outstanding = ordered - dispatched.

8. Calculate net shipped = dispatched - returned.

9. Display order status, inventory, and data-quality issues.

5. Flowchart



6. Pseudocode

BEGIN

INPUT customer_name

INPUT number_of_products

```
subtotal = 0
total_tax = 0

FOR i = 1 TO number_of_products DO

  INPUT product_name
  INPUT category
  INPUT price
  INPUT quantity

  item_total = price * quantity
  subtotal = subtotal + item_total

  IF category = "Grocery" THEN
    tax_rate = 5%
  ELSE IF category = "Electronics" THEN
    tax_rate = 18%
  ELSE IF category = "Clothing" THEN
    tax_rate = 12%
  ELSE
    tax_rate = 0%
  END IF

  item_tax = item_total * tax_rate / 100
  total_tax = total_tax + item_tax
```

```
    STORE product_name, quantity, price,
        item_total, tax_rate, item_tax

  END FOR

  IF subtotal >= 5000 THEN
    discount_rate = 20%
  ELSE IF subtotal >= 3000 THEN
    discount_rate = 10%
  ELSE IF subtotal >= 1000 THEN
    discount_rate = 5%
  ELSE
    discount_rate = 0%
  END IF

  discount = subtotal * discount_rate / 100

  maximum_concession = 1000

IF discount > maximum_concession THEN
    discount = maximum_concession
  END IF

  final_bill = subtotal - discount + total_tax
```

```
    DISPLAY "========== ITEMIZED INVOICE =========="
    DISPLAY customer_name
    DISPLAY product-wise details
    DISPLAY subtotal
    DISPLAY total_tax
    DISPLAY discount
    DISPLAY final_bill

    DISPLAY "========== THANK YOU =========="

END
```

6. Python Source Code

```
def get_tax_rate(category):
        tax_rates = {
            "grocery": 5,
            "electronics": 18,
            "clothing": 12,
            "stationery": 5
 }
return tax_rates.get(category.lower(), 0)
def calculate_discount(subtotal):
       if subtotal >= 5000:
           discount_rate = 20
       elif subtotal >= 3000:
```

```
        discount_rate = 10
    elif subtotal >= 1000:
        discount_rate = 5
    else:
        discount_rate = 0
discount = subtotal * discount_rate / 100

# Maximum concession limit
maximum_concession = 1000
discount = min(discount, maximum_concession)

return discount, discount_rate


def main():
    print("=" * 65) print(" RETAIL BILLING SYSTEM")
    print(" Multiple Discount Rules and Category Taxes")
    print("=" * 65)
    customer_name = input("Enter customer name: ")
    n = int(input("Enter number of products: "))

    products = []
    subtotal = 0
    total_tax = 0

    for i in range(n):
        print(f"\nEnter details for Product {i + 1}")

        name = input("Product name: ")
        category = input(
         "Category (Grocery/Electronics/Clothing/Stationery): "
    )
```

```
    price = float(input("Price per unit: Rs. "))
    quantity = int(input("Quantity: "))

    if price < 0 or quantity <= 0:
        print("Invalid price or quantity. Please try again.")
        return

    item_total = price * quantity
    tax_rate = get_tax_rate(category)
    item_tax = item_total * tax_rate / 100

    subtotal += item_total
    total_tax += item_tax

    products.append({
        "name": name,
        "category": category,
        "price": price,
        "quantity": quantity,
        "amount": item_total,
        "tax_rate": tax_rate,
        "tax": item_tax
    })

discount, discount_rate = calculate_discount(subtotal)
final_bill = subtotal - discount + total_tax

# Display itemized invoice
print("\n" + "=" * 100)
print("                          ITEMIZED INVOICE")
print("=" * 100)
print("Customer Name:", customer_name)
print("-" * 100)

print(
    f"{'Product':<15}{'Category':<15}{'Qty':>5}"
    f"{'Price':>12}{'Amount':>12}{'Tax %':>9}{'Tax':>12}"
)
print("-" * 100)
```

```
for p in products:
    print(
        f"{p['name']:<15}{p['category']:<15}"
        f"{p['quantity']:>5}{p['price']:>12.2f}"
        f"{p['amount']:>12.2f}{p['tax_rate']:>8.1f}%"
        f"{p['tax']:>12.2f}"
    )

print("-" * 100)
print(f"{'Subtotal:':>70} Rs. {subtotal:>10.2f}")
print(f"{'Discount rate:':>70} {discount_rate:>9.1f}%")
print(f"{'Discount amount:':>70} Rs. {discount:>10.2f}")
print(f"{'Total tax:':>70} Rs. {total_tax:>10.2f}")
print("-" * 100)
print(f"{'FINAL BILL:':>70} Rs. {final_bill:>10.2f}")
print("=" * 100)
print("             Thank you for shopping!")
print("=" * 100)
```

if **name** == "**main**": main()

7. Output Screenshot

```
Enter customer name: Lekhana
Enter number of products: 2

Enter details for Product 1
Product name: Rice
Category (Grocery/Electronics/Clothing/Stationery): Grocery
Price per unit: Rs. 1000
Quantity: 2

Enter details for Product 2
Product name: Headphones
Category (Grocery/Electronics/Clothing/Stationery): Electronics
Price per unit: Rs. 2000
Quantity: 2

================================================================================
============
                        ITEMIZED INVOICE
================================================================================
============
Customer Name: Lekhana
--------------------------------------------------------------------------------
------------
Product        Category         Qty       Price      Amount    Tax %         Tax
--------------------------------------------------------------------------------
------------
Rice           Grocery            2     1000.00     2000.00     5.0%      100.00
Headphones     Electronics        2     2000.00     4000.00    18.0%      720.00
--------------------------------------------------------------------------------
------------
                                                        Subtotal: Rs.    6000.00
                                                  Discount rate:      20.0%
                                                Discount amount: Rs.    1000.00
                                                     Total tax: Rs.     820.00
--------------------------------------------------------------------------------
------------
                                                    FINAL BILL: Rs.    5820.00
================================================================================
============
             Thank you for shopping!
================================================================================
============
```

8. Test Cases

1. ORD101: 60 + 40 dispatched against 100 ordered -> accepted.

2. ORD103: dispatch 90 against 80 ordered -> rejected as inconsistent.

3. ORD999 return: unknown order ID -> rejected.

4. ORD101 return 5 against 100 dispatched -> accepted.

5. Outstanding = ordered quantity - accepted dispatched quantity.

6. Inventory = starting stock - accepted dispatches + accepted returns.

9. Conclusion

The program validates dispatch and return records, calculates outstanding and net-shipped quantities, updates inventory consistently, and highlights invalid or inconsistent records for review.